



Active Class #4...

Maestría en Inteligencia Artificial Aplicada
SUMO Interface y Evasión de Obstáculos
Dr. David Antonio-Torres

Objetivo

Revisar el principio de funcionamiento de SUMO interface y hacer una breve introducción a los algoritmos de evasión de obstáculos





Active class

SUMO Interface

SUMO Interface



<https://sumo.dlr.de/docs/sumo-gui.html>

SUMO (Simulation of Urban Mobility) es una interfaz para la simulación de tráfico microscópico.

La simulación de tráfico microscópico se enfoca en vehículos individuales y su interacción. Modela el comportamiento de cada vehículo, incluyendo aceleración, desaceleración, cambio de carril, etc.

SUMO Interface: Instalación



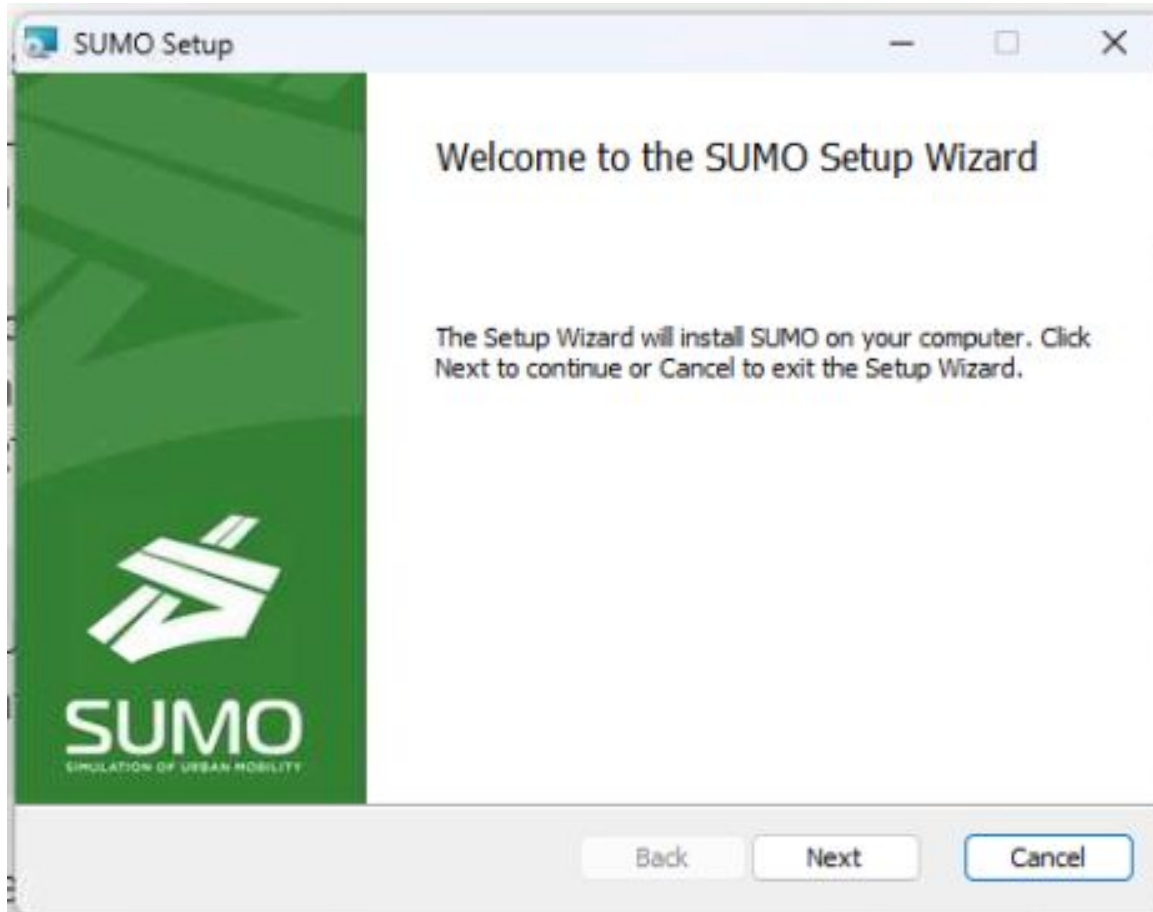
Es necesario visitar el siguiente enlace y elegir uno de los instaladores (se debe usar el instalador):

<https://sumo.dlr.de/docs/Installing/index.html#windows>

- Download 64-bit installer: [sumo-win64-1.27.0.msi](#)
- Download 64-bit zip: [sumo-win64-1.27.0.zip](#)
- Download 64-bit installer with all extras (contains GPL code): [sumo-win64extra-1.27.0.msi](#)
- Download 64-bit zip with all extras (contains GPL code): [sumo-win64extra-1.27.0.zip](#)

<https://sumo.dlr.de/docs/sumo-gui.html>

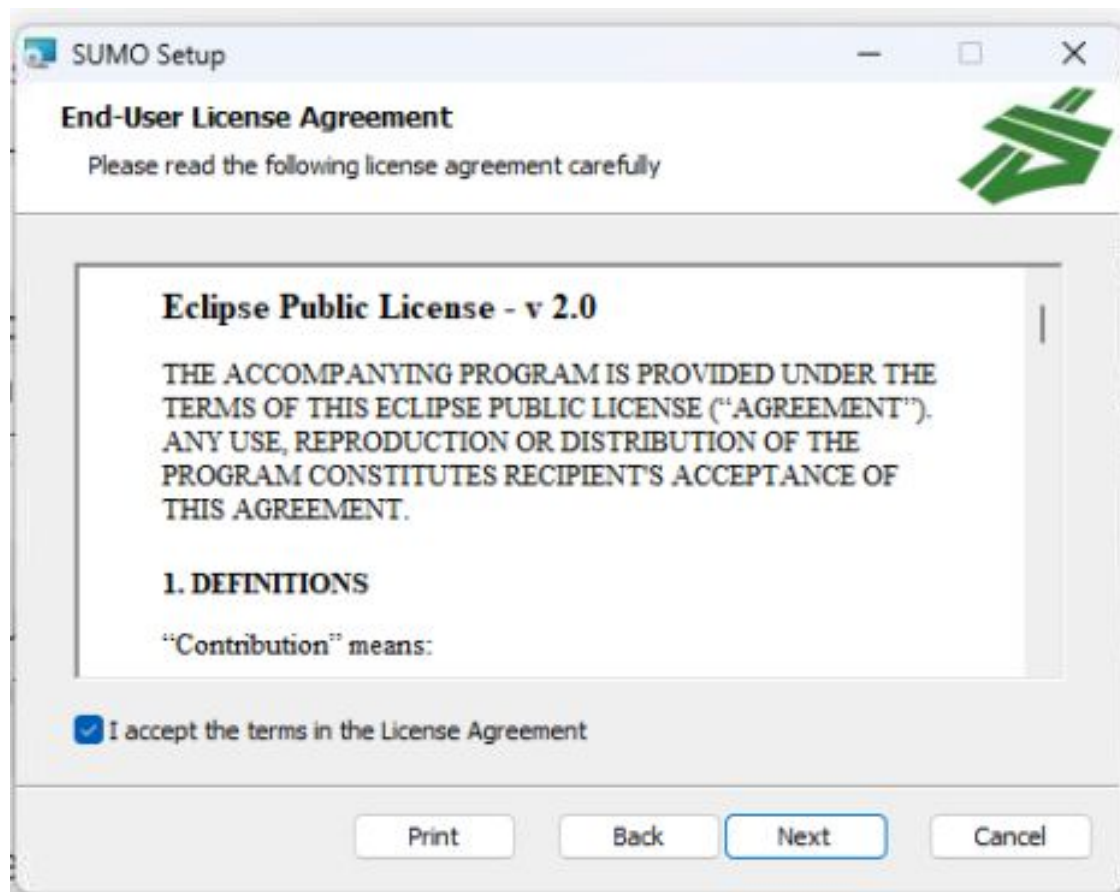
SUMO Interface: Instalación



Para los instaladores MSI, se procede haciendo doble clic en el instalador.

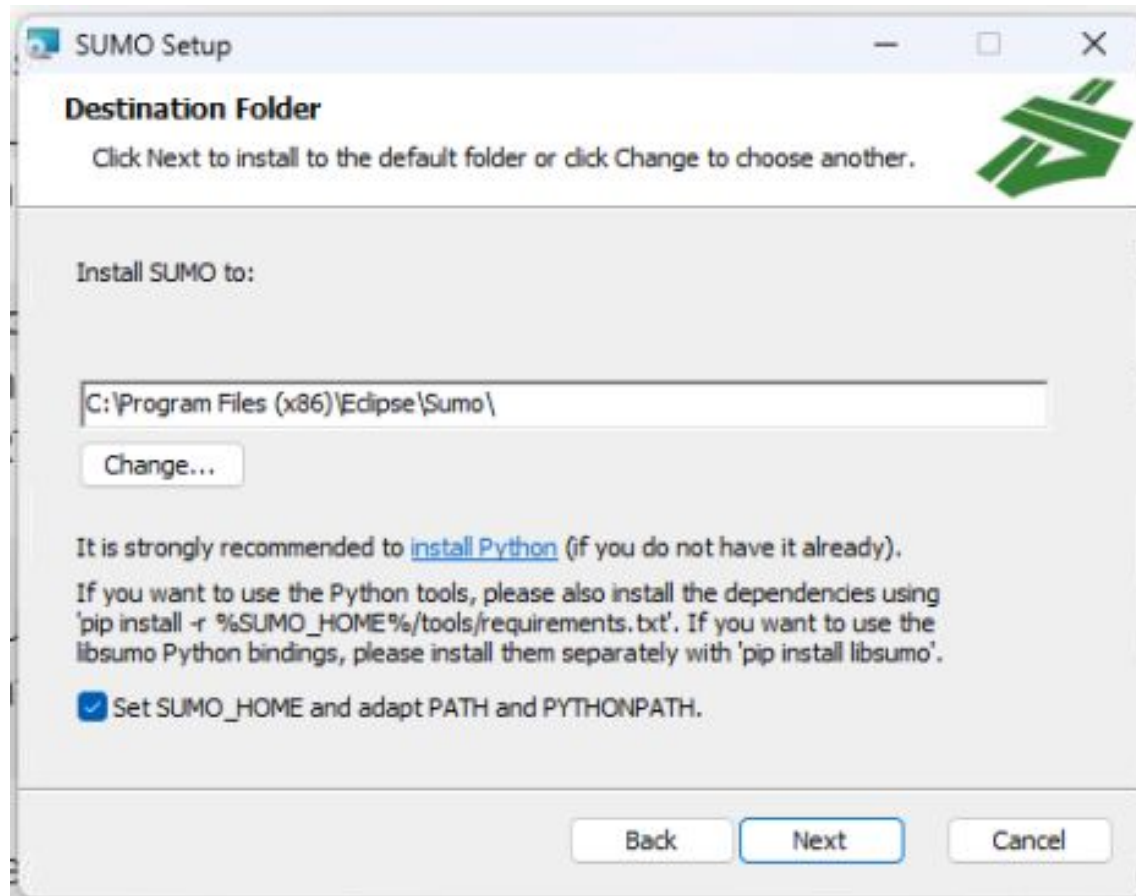
En la ventana de bienvenida, se pulsa el botón Next.

SUMO Interface: Instalación



En la ventana de EULA, aceptamos los términos de la licencia y pulsamos el botón Next.

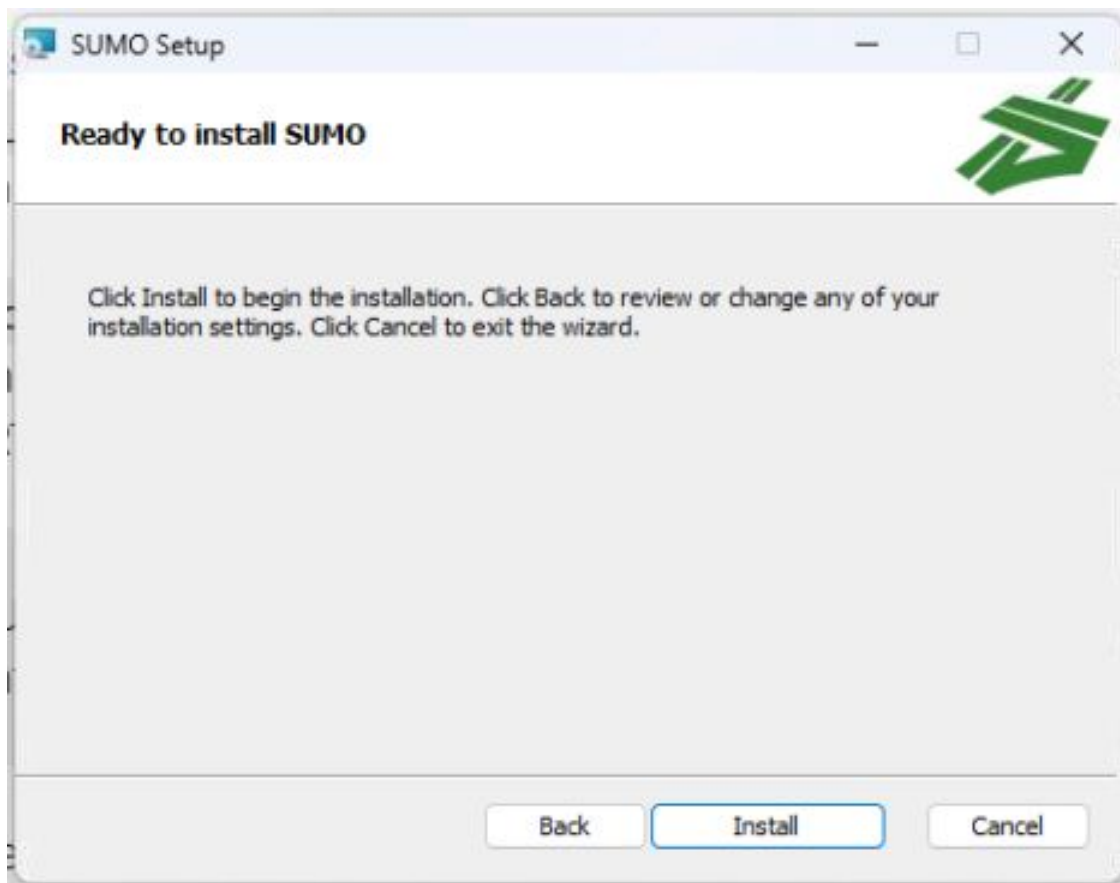
SUMO Interface: Instalación



En la ventana para seleccionar la carpeta destino, mantenemos la ruta por omisión y pedimos que el PATH y el PYTHONPATH sea actualicen con la variable SUMO_HOME. Dicha variable se inicializa con la ruta seleccionada.

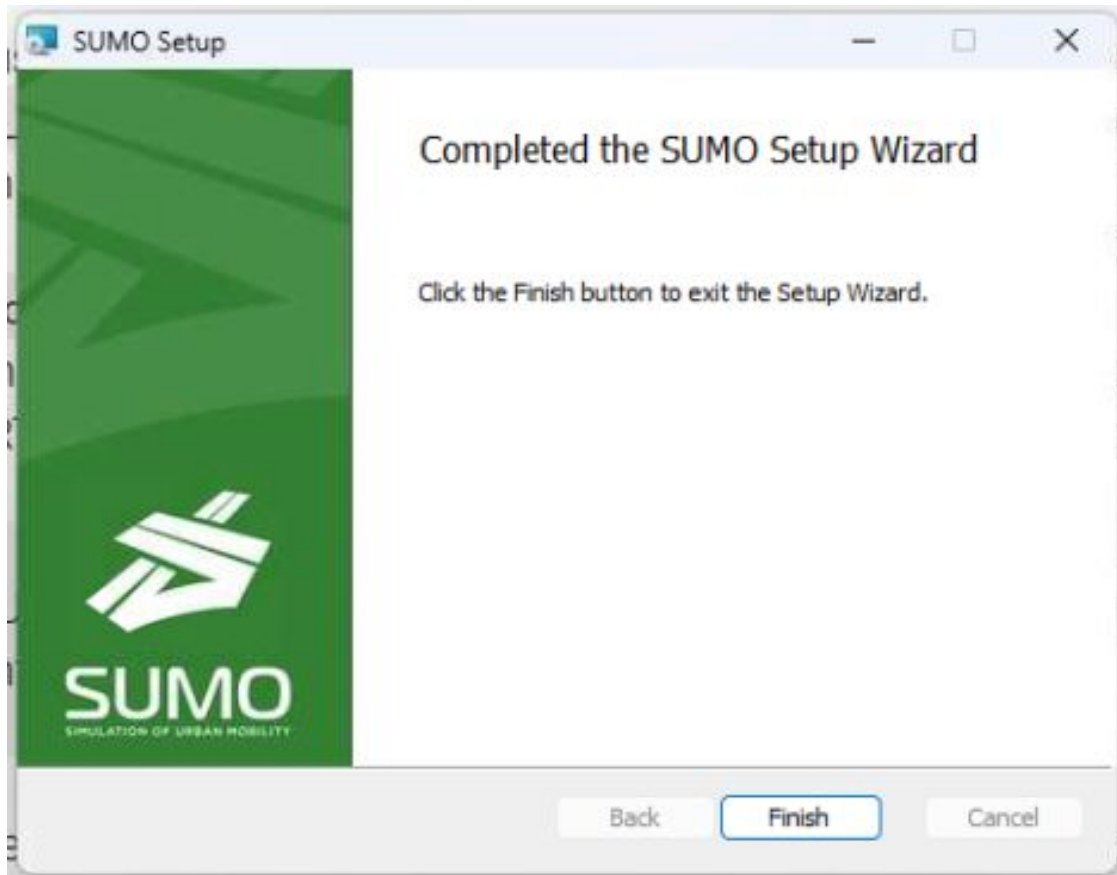
Pulsamos el botón Next.

SUMO Interface: Instalación



En la ventana de instalación, pulsamos el botón Install para proceder con la instalación.

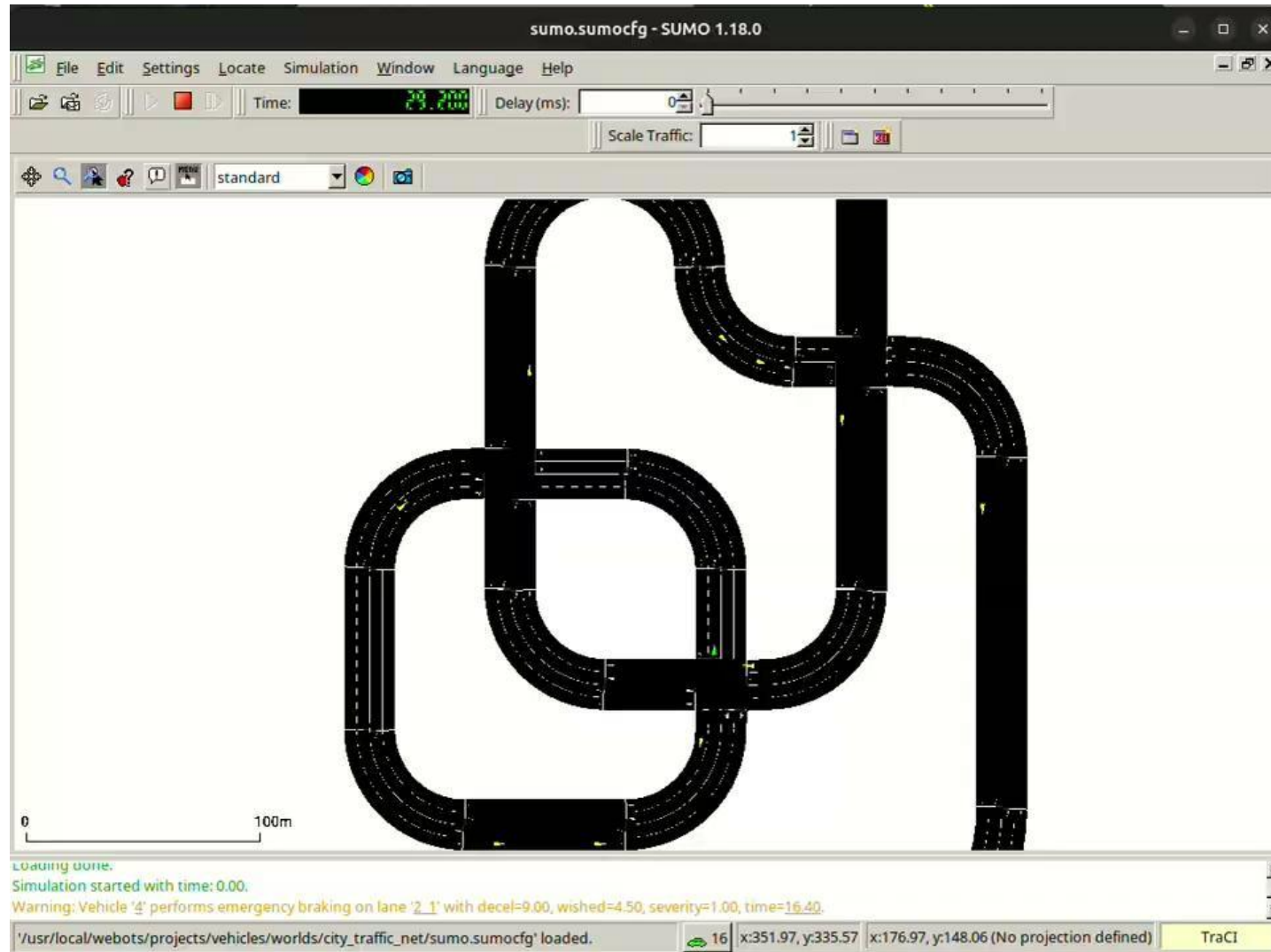
SUMO Interface: Instalación



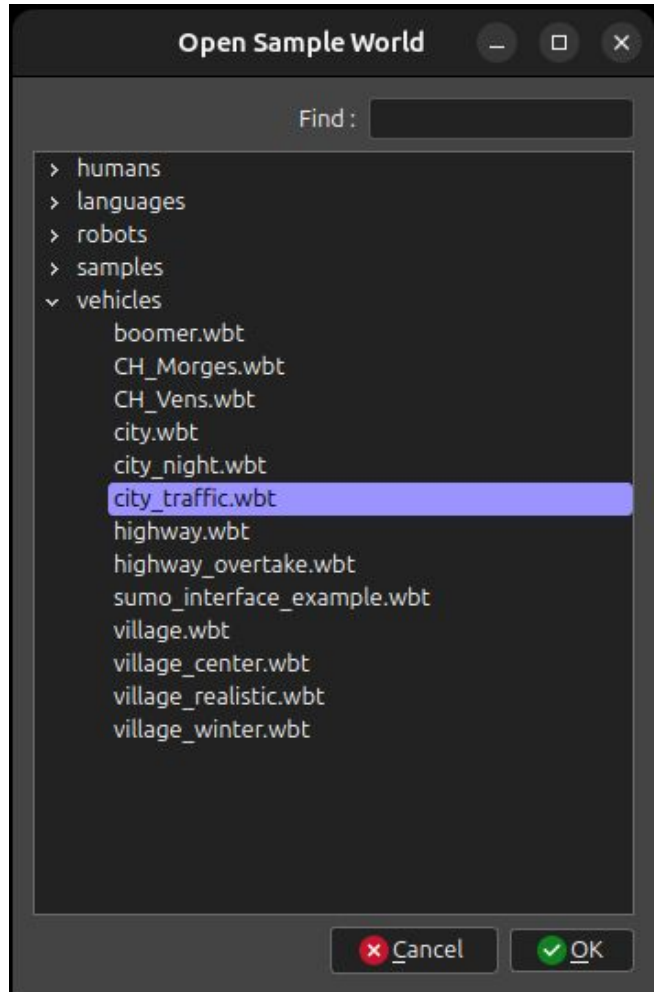
Después de unos minutos, recibimos el mensaje de que la instalación ha concluido. Pulsamos el botón Finish.

No es necesario reiniciar la PC para proceder a validar la instalación de SUMO Interface.

SUMO Interface



SUMO Interface

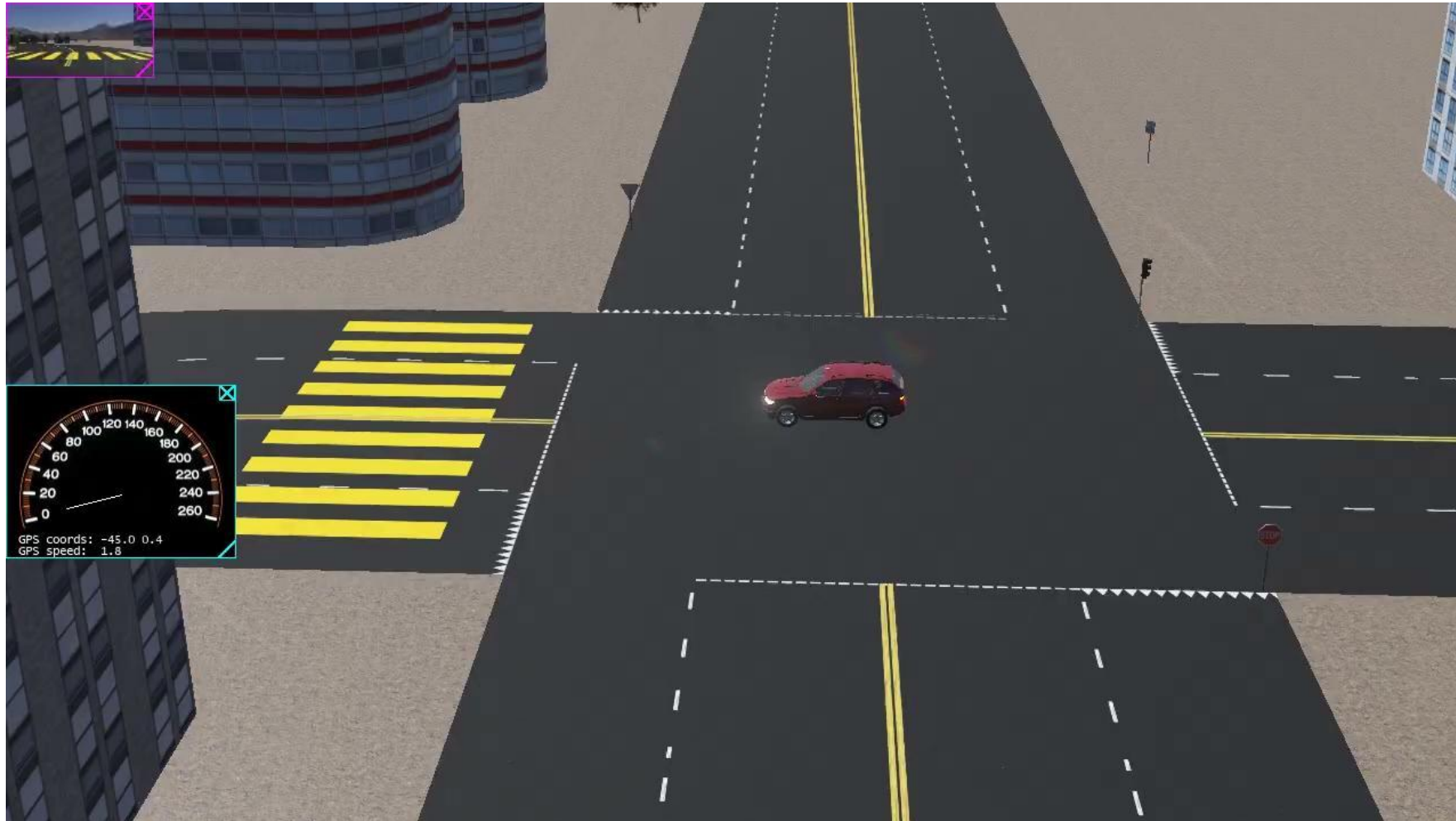


Uno de los sample worlds de Webots que hace uso de SUMO Interface es el City Traffic.

Este mundo requiere que SUMO se encuentre ya instalado y con la PATH actualizada para proceder a la simulación de tráfico.

Aquellos mundos de Webots que hacen uso de SUMO aumentan el número de vehículos simulados conforme progresa la simulación. Esto incrementa los requerimientos de procesamiento.

SUMO Interface





Active class

Evasión de Obstáculos

Evación de Obstáculos



La evasión de obstáculos en robótica es la capacidad de un sistema autónomo de detectar y maniobrar alrededor de una barrera física que obstruye su ruta.

La evasión de obstáculos puede conducir a una planeación de trayectorias (global planner) o una evasión reactiva (local planner)

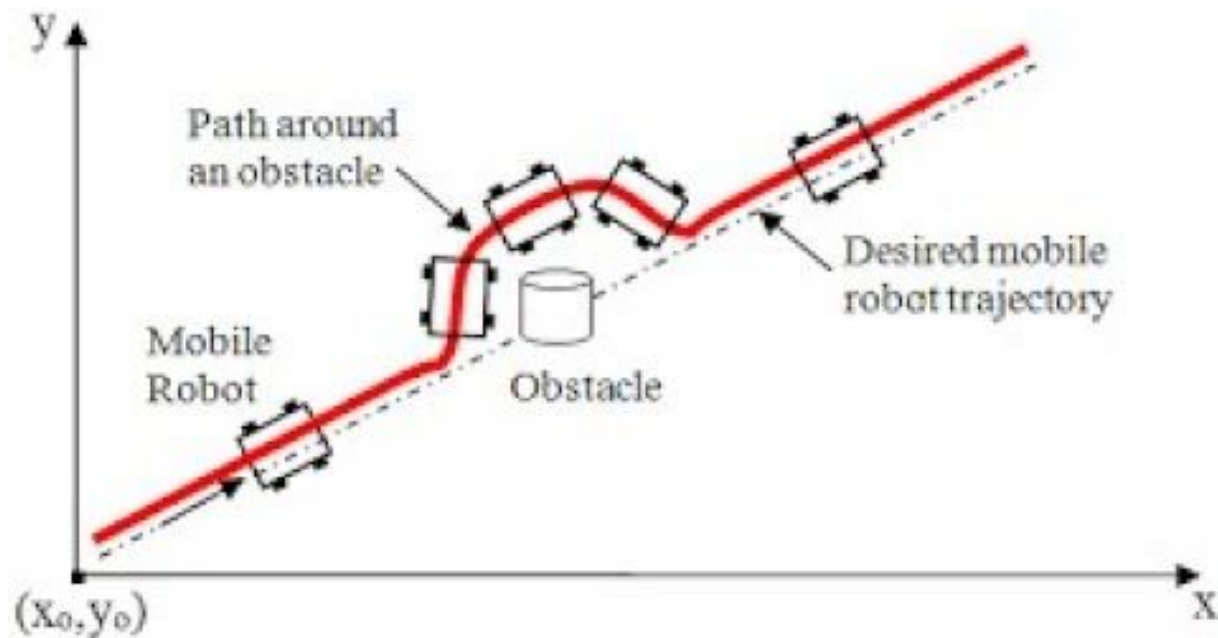
Planeación de Trayectorias



La planeación de trayectorias en robótica es el proceso de encontrar una ruta óptima, segura y libre de colisiones que le permita a un robot ir desde un punto inicial a un punto destino.

Algunos algoritmos de planeación de trayectorias se basan en mapas estáticos previamente definidos.

Evación Reactiva

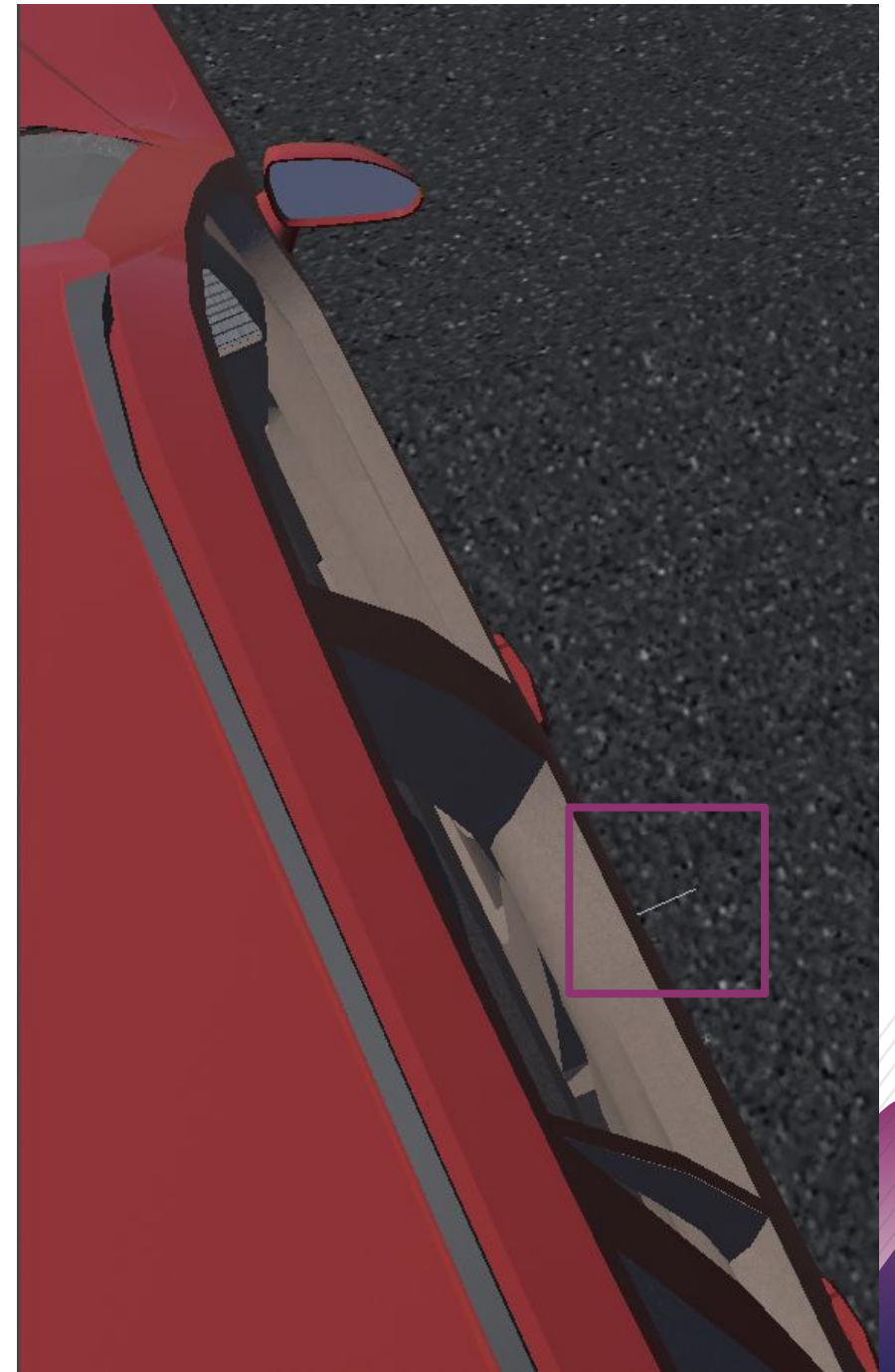


La evasión reactiva en robótica es una estrategia de navegación inmediata que la permite a un robot detectar un obstáculo no previsto y maniobrar para lograr la evasión. Todo esto en tiempo real.

Evasión Reactiva



Evación Reactiva

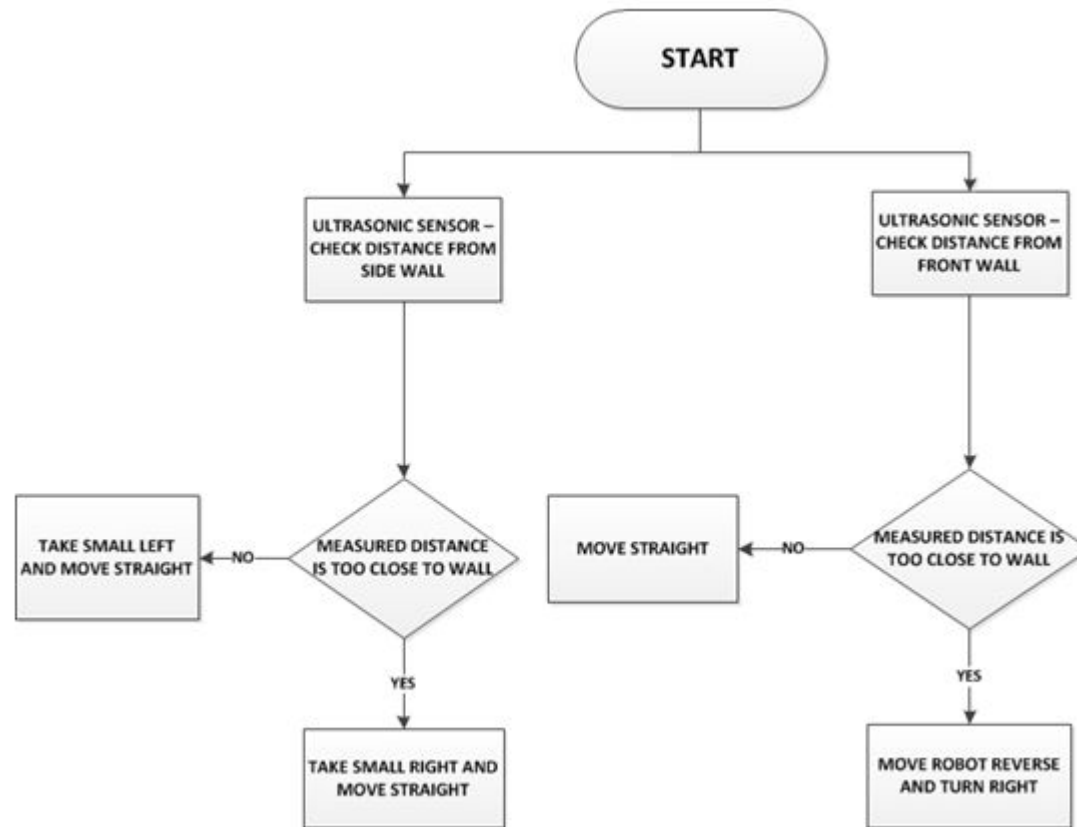




Active class

Robot Seguidor de Pared

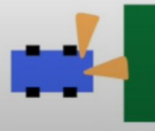
Robot Seguidor de Pared



Es un robot móvil diseñado para navegar a lo largo de una pared, detectando su presencia y ajustando su movimiento de acuerdo al escenario.

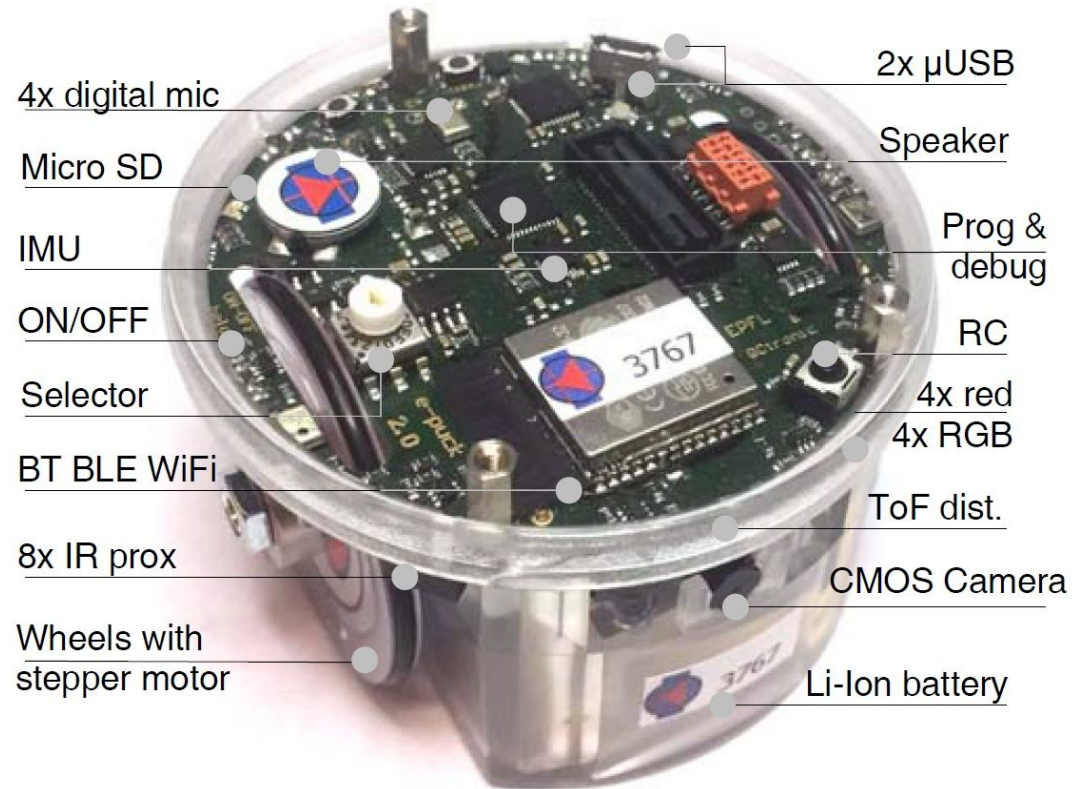
El robot puede ser seguidor de pared por la derecha o por la izquierda.

Robot Seguidor de Pared

Left Wall	Front Wall	Direction	
✓	✗	Forward	
✓	✓	Turn right	
✗	✗	Turn left	
✗	✓	Turn right	

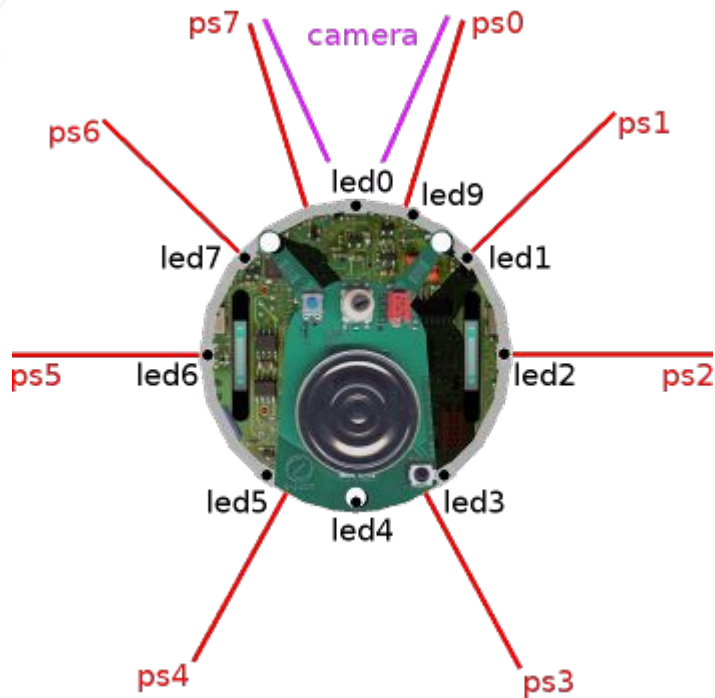
El algoritmo de seguimiento de pared por la izquierda puede simplificarse con los pasos y condiciones indicadas.

Robot E-puck



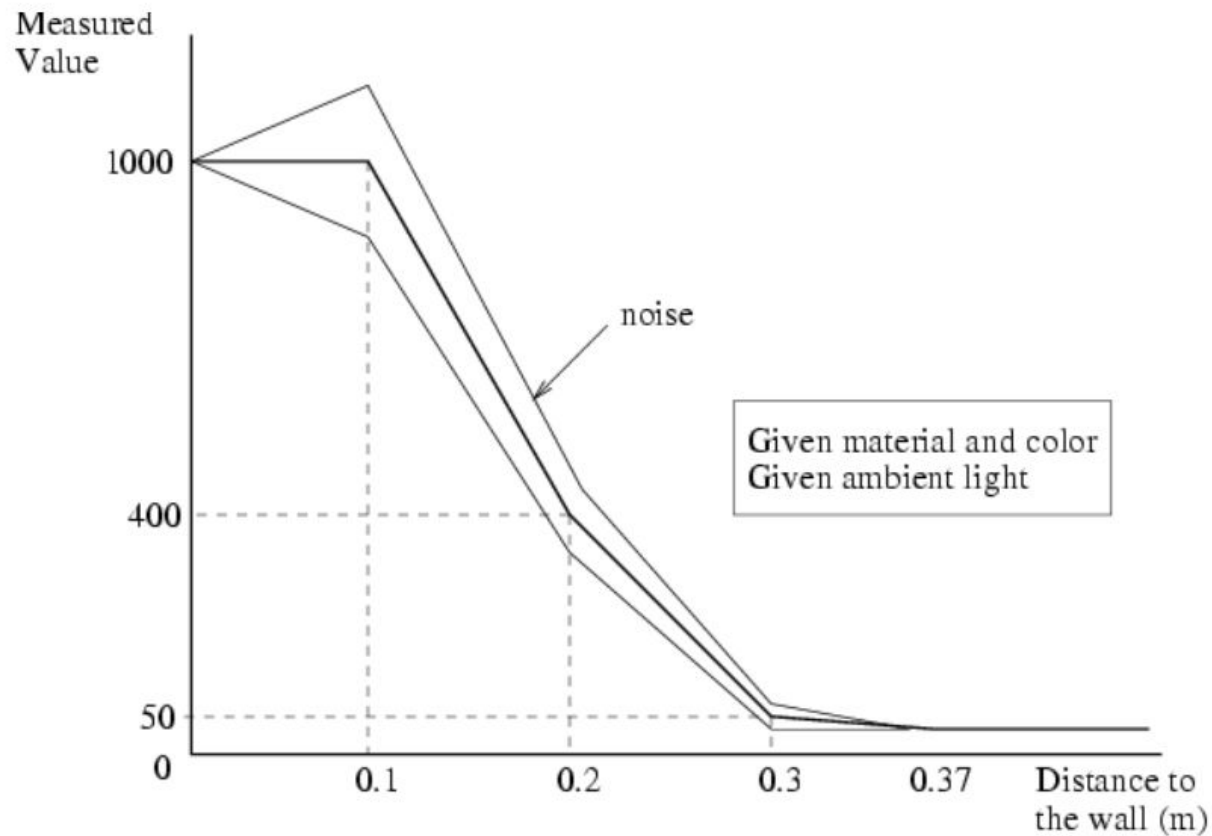
Para poder llevar a cabo el seguimiento de pared, el robot usualmente se equipa con uno o más de estos sensores: LiDAR, Radar o ultrasónico.

Robot E-puck



Device	Name
Motors	'left wheel motor' and 'right wheel motor'
Position sensors	'left wheel sensor' and 'right wheel sensor'
Proximity sensors	'ps0' to 'ps7'
Light sensors	'ls0' to 'ls7'
LEDs	'led0' to 'led7' (e-puck ring), 'led8' (body) and 'led9' (front)
Camera	'camera'
Accelerometer	'accelerometer'
Gyro	'gyro'
Ground sensors (extension)	'gs0', 'gs1' and 'gs2'
Speaker	'speaker'

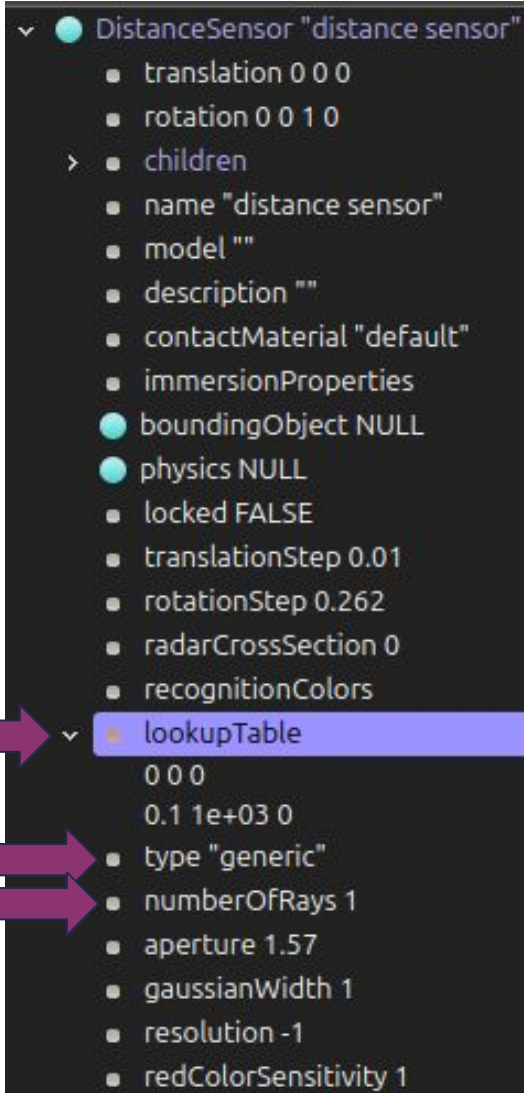
Sensor de Distancia



La relación entre distancia y lectura del sensor se plasma a través de su lookup table, la cual es editable. La primera columna es la distancia, la segunda columna es el valor de lectura y la tercera columna es la desviación estándar del ruido agregado. Distancias y valores no proporcionados son interpolados.

```
lookupTable [ 0      1000  0,  
              0.1    1000  0.1,  
              0.2     400  0.1,  
              0.3      50  0.1,  
              0.37     30  0  ]
```

Sensor de Distancia



La relación entre distancia y lectura del sensor se plasma a través de su lookup table, la cual es editable. La primera columna es la distancia, la segunda columna es el valor de lectura y la tercera columna es la desviación estándar del ruido agregado. Distancias y valores no proporcionados son interpolados.

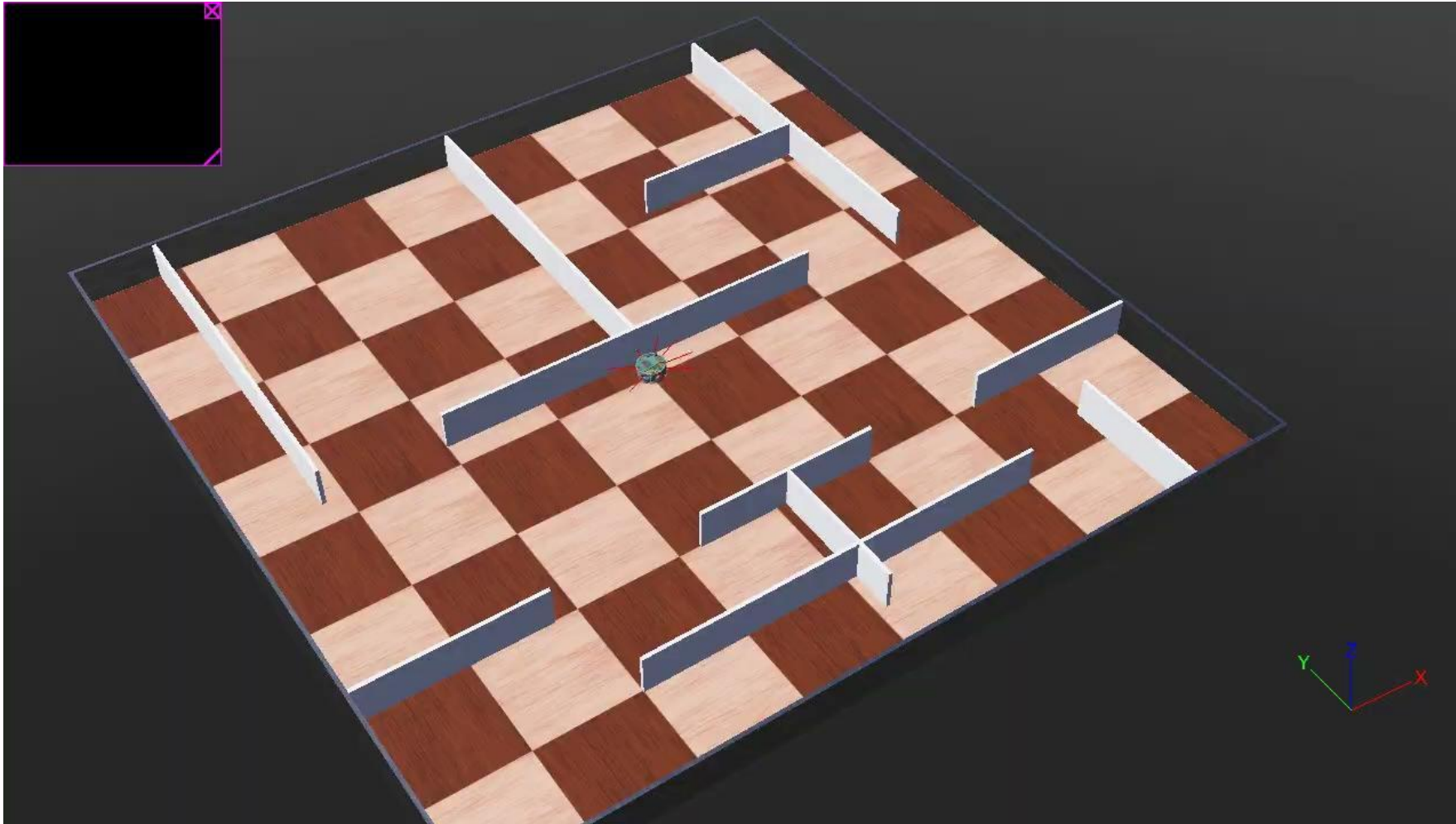
```
lookupTable [ 0      1000  0,  
              0.1    1000  0.1,  
              0.2    400   0.1,  
              0.3    50    0.1,  
              0.37   30    0   ]
```

Sensor de Distancia

Webots permite diferentes tipos de sensores asociados al sensor de distancia.

type (field)	"generic"	"infra-red"	"sonar"	"laser"
numberOfRays (field)	> 0	> 0	> 0	1
Distance calculation	Average	Average	Nearest	Nearest
gaussianWidth (field)	Used	Used	Ignored	Ignored
Sensitive to red objects	No	Yes	No	No
Draws a red spot	No	No	No	Yes
Ignore transparent objects	No	Yes	No	Yes

Robot Seguidor de Línea



Robot Seguidor de Línea

```
my_controller_wall_follower.py x
4 # from controller import Robot, Motor, DistanceSensor
5 from controller import Robot
6
7 max_speed = 6.28
8
9 # create the Robot instance.
10 robot = Robot()
11
12 # get the time step of the current world.
13 timestep = int(robot.getBasicTimeStep())
14
15 #Motors
16 leftMotor = robot.getDevice('left wheel motor')
17 rightMotor = robot.getDevice('right wheel motor')
18
19 leftMotor.setPosition(float('inf'))
20 rightMotor.setPosition(float('inf'))
21
22 leftMotor.setVelocity(0.0)
23 rightMotor.setVelocity(0.0)
24
25 #proximity sensors
26 prox_sensors = []
27 for ind in range(8):
28     sensor_name = 'ps' + str(ind)
29     prox_sensors.append(robot.getDevice(sensor_name))
30     prox_sensors[ind].enable(timestep)
31
```

```
my_controller_wall_follower.py x
31
32 # Main loop:
33 # - perform simulation steps until Webots is stopping the controller
34 while robot.step(timestep) != -1:
35     # Read the sensors:
36     for ind in range(5,8):
37         print("ind:{}, val:{}".format(ind, prox_sensors[ind].getValue()))
38
39     #process sensor data
40     left_wall = prox_sensors[5].getValue() > 75
41     front_wall = prox_sensors[7].getValue() > 75
42     left_corner = prox_sensors[6].getValue() > 75
43
44     left_speed = max_speed
45     right_speed = max_speed
46
47     if front_wall:
48         print("Turning right")
49         left_speed = max_speed
50         right_speed = -max_speed
51
52     elif left_wall:
53         print("Driving forward")
54         left_speed = max_speed
55         right_speed = max_speed
56
57     elif left_corner:
58         print("Too close to wall, driving right")
59         left_speed = max_speed
60         right_speed = max_speed/8
61
62     else:
63         print("Turning left")
64         left_speed = max_speed/8
65         right_speed = max_speed
66
67     leftMotor.setVelocity(left_speed)
68     rightMotor.setVelocity(right_speed)
69
70
```



Active class

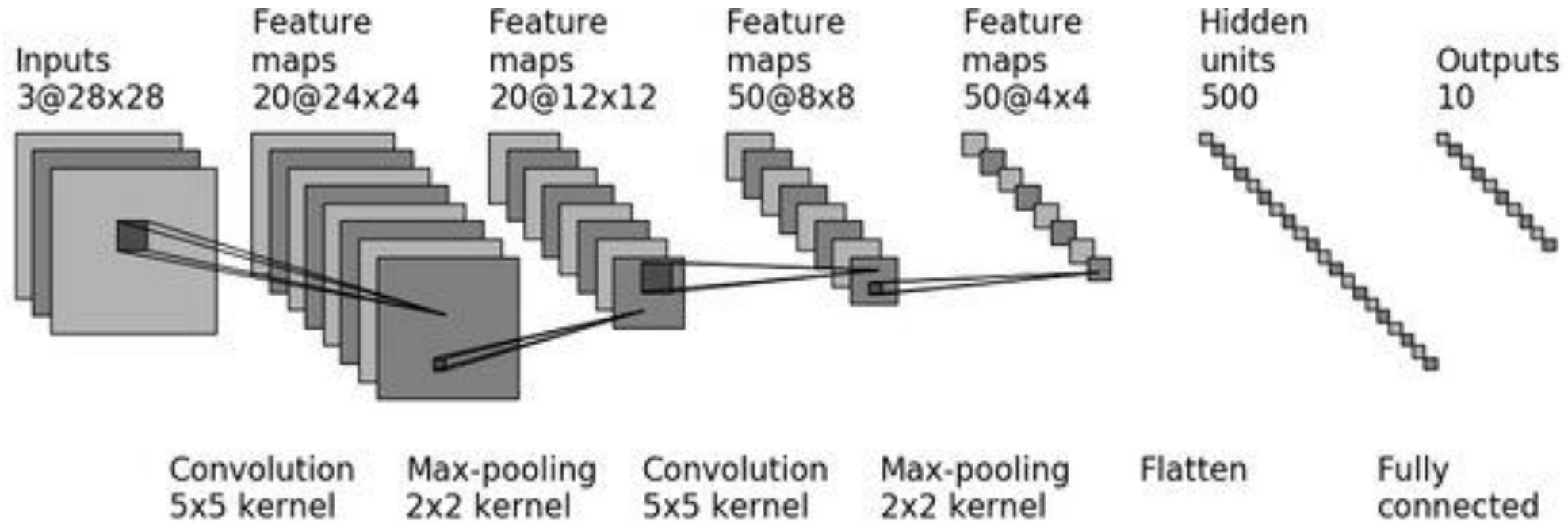
Componentes del Proyecto Final



Active class

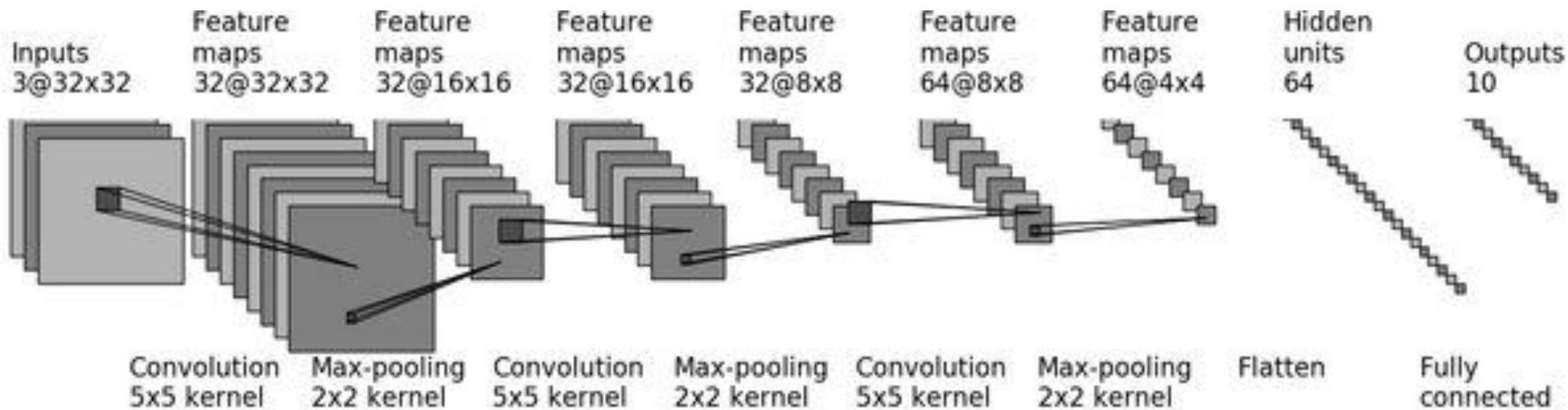
Arquitecturas CNN

CNN - LeNet5



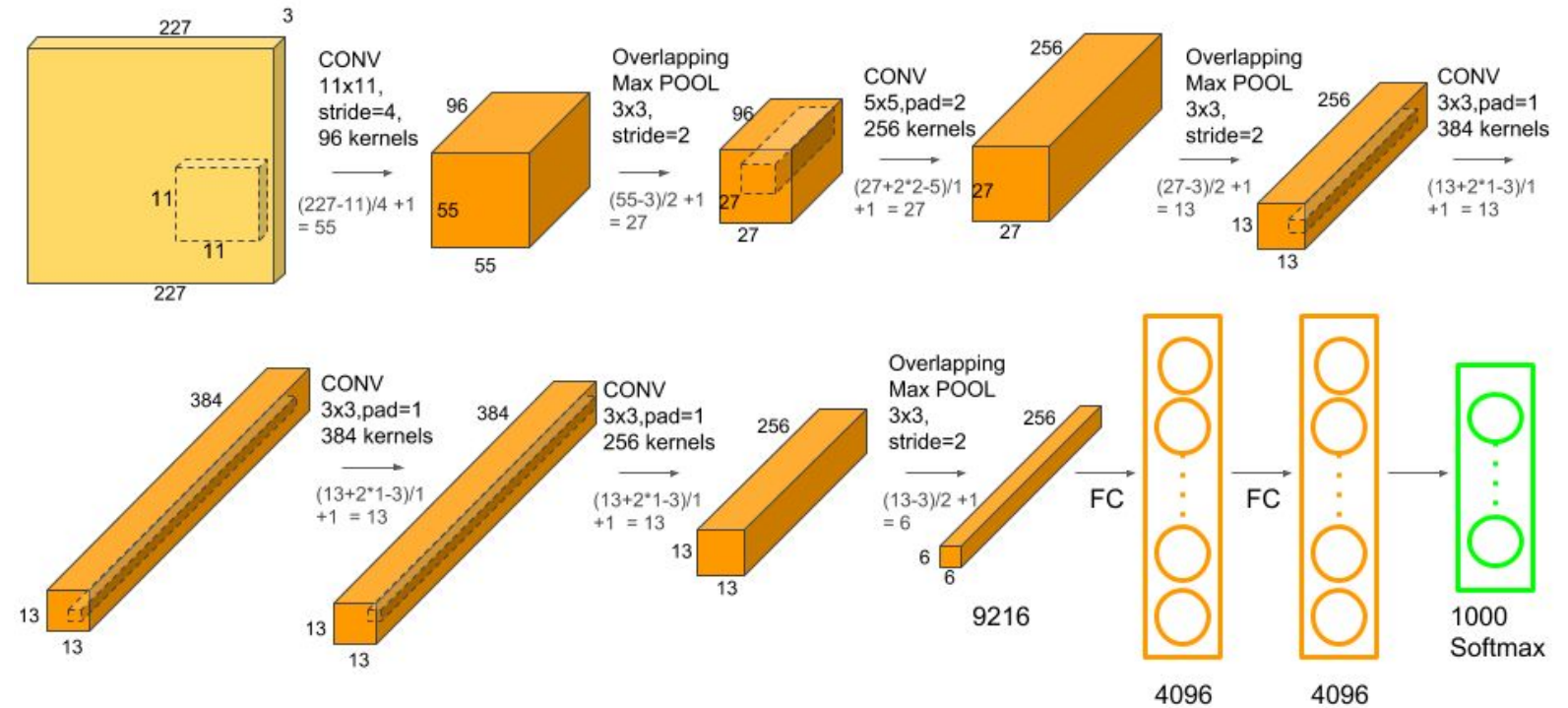
https://www.researchgate.net/publication/312170477_On_Classification_of_Distorted_Images_with_Deep_Convolutional_Neural_Networks/figures?lo=1

CNN - CIFAR10



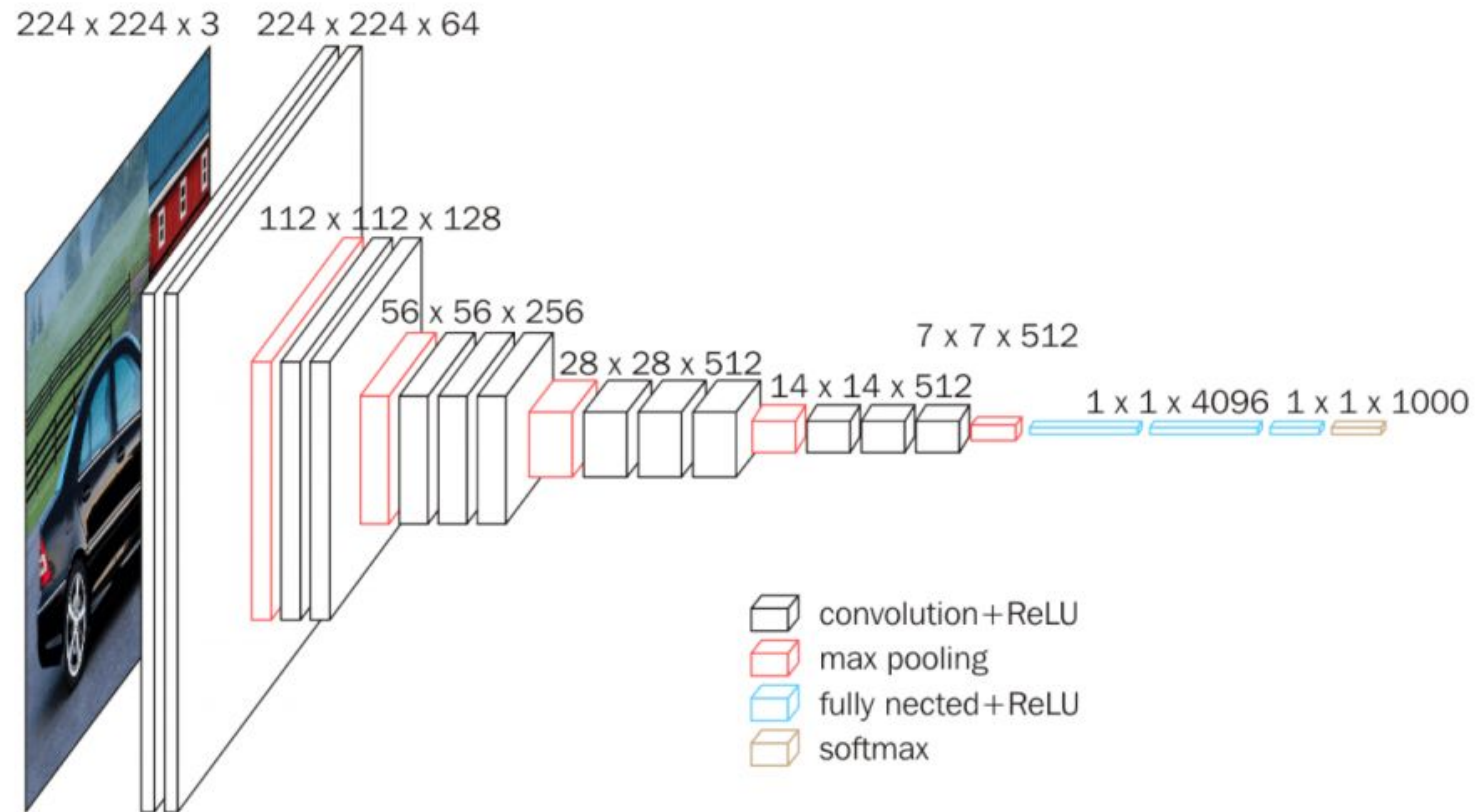
https://www.researchgate.net/publication/312170477_On_Classification_of_Distorted_Images_with_Deep_Convolutional_Neural_Networks/figures?lo=1

CNN - AlexNet



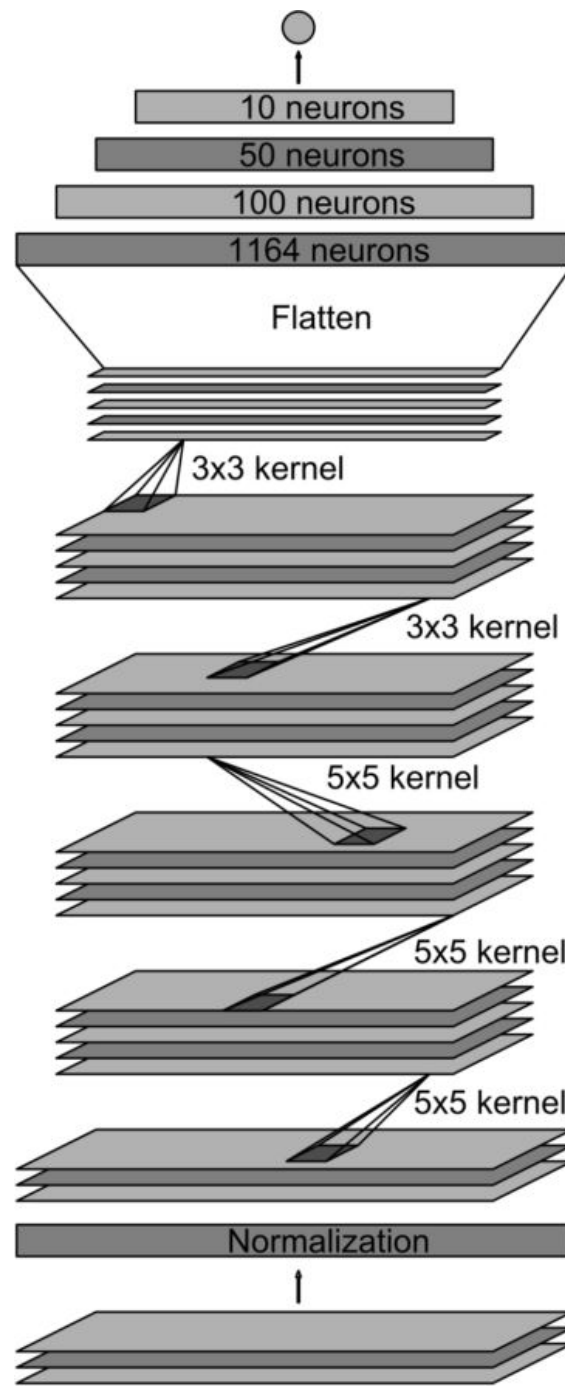
<https://blog.paperspace.com/popular-deep-learning-architectures-alexnet-vgg-googlenet/>

CNN - VGG16



<https://blog.paperspace.com/popular-deep-learning-architectures-alexnet-vgg-googlenet/>

CNN - Nvidia Model



Output: vehicle control

Fully-connected layer
Fully-connected layer
Fully-connected layer

Convolutional
feature map
64@1x18

Convolutional
feature map
64@3x20

Convolutional
feature map
48@5x22

Convolutional
feature map
36@14x47

Convolutional
feature map
24@31x98

Normalized
input planes
3@66x200

Input planes
3@66x200

<https://developer.nvidia.com/blog/deep-learning-self-driving-cars/>



Active class

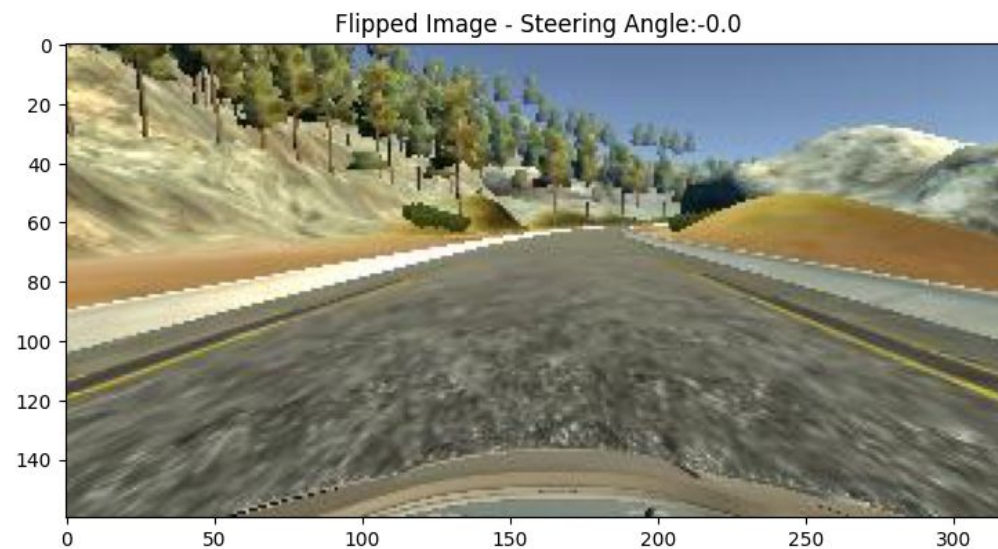
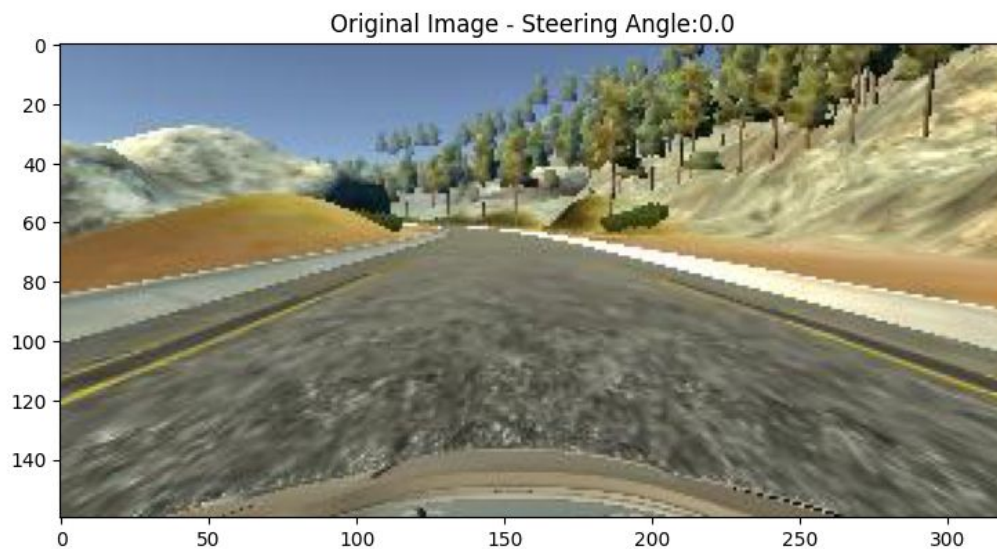
Data Augmentation

Data Augmentation

<https://imgaug.readthedocs.io/en/latest/>

<https://towardsdatascience.com/image-augmentation-with-skimage-python-7f53f1b0eeb5>

Es el proceso de las muestras en el dataset creando nuevas imágenes a partir de las ya existentes. Se puede usar **ImageDataGenerator** de Keras, **Scikit-image** o la biblioteca **imgaug**. Rotación, zoom, brightness, flip, etc. son algunas operaciones adecuadas para este paso.





Active class

Sensores en el Proyecto Final

Uso de Cámara

- Vamos usar un mundo en Webots sin tráfico ni peatones
- Vamos a grabar imágenes mientras conducimos el vehículo autónomo por el mundo usando el teclado
- Cada imagen guardada deberá tener un nombre particular, a la cual se le va a asociar el ángulo de conducción (Imitation Learning)
- Un archivo CSV mostrará un listado de las imágenes guardadas y el ángulo de conducción correspondiente
- Esta información será usada para entrenar una CNN
- El modelo resultante será usado para lograr una conducción autónoma

Uso de Otros Sensores

- Vamos a usar un segundo mundo en Webots con tráfico, peatones y vehículos estacionados
- Los peatones tendrán sus respectivos controladores en Python y trayectorias y velocidades definidas
- El tráfico va a ser manejado por SUMO de manera automática
- La cámara a bordo vendrá equipada con un nodo de reconocimiento de objetos (Recognition) para hacer la detección de vehículos y peatones
- Si se detecta peatón, el vehículo autónomo se detiene hasta que concluya la detección
- Si se detecta un vehículo, el vehículo autónomo continúa conduciendo a una cierta distancia (definida en código) evitando colisión usando un LiDAR; si la distancia se vuelve menor a la definida, se asumirá que el vehículo se encuentra estacionado y se procede a la evasión de obstáculos usando sensores de distancia
- Un giroscopio permitirá al vehículo retornar a su postura (pose) original
- Se hará uso de un GPS para tomar decisiones en entronques (forks) cuando existe más de una opción de conducción (Conditional Imitation Learning)



Active class

Referencias

Referencias

- <https://sumo.dlr.de/docs/sumo-gui.html>
- <https://sumo.dlr.de/docs/Installing/index.html#windows>
- <https://www.vecnarobotics.com/resources/obstacle-avoidance-in-amr-and-agv-robots-explained/>
- https://www.theconstruct.ai/robotigniteacademy_learnros/ros-courses-library/path-planning-basics/
- <https://github.com/Mummanajagadeesh/obstacle-avoidance-robot-w/blob/main/README.md>
- <https://www.engineersgarage.com/efficient-wall-following-robot-with-ultrasonic-sensor-that-works-in-both-indoor-and-outdoor-environments/>
- <https://e-puck.gctronic.com/>
- <https://www.cyberbotics.com/doc/guide/epuck?version=R2021a>
- <https://cyberbotics.com/doc/reference/distancesensor>
- https://www.researchgate.net/publication/312170477_On_Classification_of_Distorted_Images_with_Deep_Convolutional_Neural_Net_works/figures?lo=1
- https://www.researchgate.net/publication/312170477_On_Classification_of_Distorted_Images_with_Deep_Convolutional_Neural_Net_works/figures?lo=1
- <https://blog.paperspace.com/popular-deep-learning-architectures-alexnet-vgg-googlenet/>
- <https://developer.nvidia.com/blog/deep-learning-self-driving-cars/>
- <https://imgaug.readthedocs.io/en/latest/>
- <https://towardsdatascience.com/image-augmentation-with-skimage-python-7f53f1b0eeb5>



D.R.© Tecnológico de Monterrey, México, 2023.
Prohibida la reproducción total o parcial
de esta obra sin expresa autorización del
Tecnológico de Monterrey.